DevOps Scenario-Based Interview Q&A

0–2 Years Experience | Top MNCs & Product Companies

20 Questions with Structured Answers

Q01 Your application is down in production. How will you troubleshoot?

■ Immediate Triage

- Check monitoring dashboards (Datadog/Grafana) for recent alerts
- Verify if the issue is full outage or partial (check health endpoints)
- Identify when it went down — correlate with recent deployments

■ System Layer Check

- EC2/Node status: is the instance running? Check cloud console
- Application process: is the app running? (systemctl status / docker ps / kubectl get pods)
- Port listening: netstat -tlnp | grep

■ Log Analysis

- App logs: kubectl logs --tail=100 / docker logs
- System logs: journalctl -u -n 100
- Look for OOMKilled, connection refused, 500 errors

■ Network & Dependencies

- Check DB connectivity, external API availability
- Verify DNS resolution and load balancer targets are healthy

■ Resolution

- If recent deploy: rollback immediately
- If OOM: increase memory limits or fix memory leak
- Notify stakeholders; document timeline for post-mortem

■ *Interview tip: Always say 'I'd check monitoring first, then correlate with recent changes' — this shows you think systematically, not randomly.*

Q02 Your server CPU suddenly spikes to 100%. What steps will you take?

■ Identify the Process

- top or htop — sort by CPU usage (%CPU column)
- ps aux --sort=-%cpu | head -10
- pidstat -u 1 5 — per-process CPU every second

■ Diagnose Root Cause

- Check if it's app-level (runaway thread, infinite loop) or system (kernel task)
- strace -p — see what the process is doing
- lsof -p — check open file descriptors

■ Kubernetes Context

- kubectl top pods --all-namespaces
- kubectl top nodes
- Check if HPA is scaling — kubectl get hpa

■ **Fix**

- If rogue process: kill -9 after investigation
- If app bug: rollback or hotfix
- If legit load spike: scale out horizontally (ASG/HPA)
- Set CPU limits in Kubernetes to prevent noisy neighbor issues

■ *Interview tip: Mention CPU limits in Kubernetes. Most freshers forget this — it shows production awareness.*

Q03 Your Docker container is crashing repeatedly. How will you debug?

■ **Check Container State**

- docker ps -a — see exit codes (137=OOM, 1=app error, 126=permission denied)
- docker inspect — check State.ExitCode and State.Error

■ **Read Logs**

- docker logs --tail=50
- docker logs 2>&1 | grep -i error

■ **Interactive Debug**

- docker run -it --entrypoint /bin/sh — bypass CMD and inspect manually
- Check if config files, env vars, or secrets are missing

■ **Resource Issues**

- docker stats — check memory and CPU usage
- If OOM: increase memory limit in docker run --memory flag

■ **Common Causes**

- Missing environment variable
- App can't connect to DB (check connection string, network)
- Permission issue on mounted volume
- Missing dependency in image (bad Dockerfile)

■ *Interview tip: Exit code 137 = OOM Kill. Memorize the top 5 exit codes — it immediately signals hands-on experience.*

Q04 Your Kubernetes pod is not coming up. What will you check?

■ Pod Status

- kubectl get pods — check STATUS column (Pending, CrashLoopBackOff, ImagePullBackOff, OOMKilled)
- kubectl describe pod — read Events section at the bottom

■ Common Status → Fix

- Pending: Node resource exhaustion — kubectl describe node | grep -A5 Allocated
- ImagePullBackOff: Wrong image tag or ECR auth issue — check imagePullSecrets
- CrashLoopBackOff: App crashing — check logs: kubectl logs --previous
- OOMKilled: Memory limit too low — increase resources.limits.memory

■ Config Issues

- kubectl get events --sort-by=.metadata.creationTimestamp
- Check ConfigMap and Secret mounts are correct
- Verify PVC is bound if pod uses persistent storage

■ Readiness/Liveness Probe

- Misconfigured probe path or port causes pod to never become Ready
- kubectl describe pod | grep -A5 Liveness

■ *Interview tip: Always say you check 'kubectl describe pod' Events section first. That single command answers 80% of pod issues.*

Q05 Your CI/CD pipeline is failing. How do you identify the issue?

■ Locate the Failure

- Open the pipeline UI (Jenkins/GitHub Actions) and find the failing stage
- Read the last 20-30 lines of build log — error is usually near the bottom

■ Common Stage Failures

- Build stage: compilation error, missing dependency, wrong JDK version
- Test stage: unit test failure — check test report for specific assertion
- Docker build: bad Dockerfile syntax, base image unavailable
- Push stage: registry auth failure — check credentials in secrets
- Deploy stage: kubectl apply error — YAML syntax, missing namespace

■ Environment Issues

- Agent/runner out of disk space or memory
- Dependency version mismatch between local and CI environment
- Network timeout fetching packages (npm, maven, pip)

■ Reproduce & Fix

- Run the failing step locally to confirm
- Check if pipeline was passing before — git bisect to find bad commit
- Fix, push to feature branch, re-run pipeline

■ *Interview tip: Say 'I look at the failing stage first, not the full log' — shows you're efficient, not someone who reads 5000 lines randomly.*

Q06 Deployment failed after release. How do you perform rollback?

■ **Kubernetes Rollback**

- kubectl rollout undo deployment/ — instant rollback to previous revision
- kubectl rollout undo deployment/ --to-revision= — rollback to specific version
- kubectl rollout status deployment/ — verify rollback succeeded
- kubectl rollout history deployment/ — see revision history

■ **Helm Rollback**

- helm history — see all revisions
- helm rollback

■ **Before Rollback Checklist**

- Confirm the issue is deployment-related, not infra or DB migration
- Check if DB schema change was part of the release — rollback may break DB compatibility
- Notify the team before executing

■ **Post-Rollback**

- Verify app is healthy: kubectl get pods + hit health endpoint
- Document what failed and why
- Fix in dev/staging before re-deploying

■ *Interview tip: Mention the DB migration risk — most freshers only talk about kubectl rollout undo. Raising the DB point shows maturity.*

Q07 Your website is slow. How do you find the root cause?

■ Frontend vs Backend

- • Browser DevTools (Network tab) — check which requests are slow
- • Time To First Byte (TTFB) high = backend/server issue
- • Large asset sizes = frontend optimization needed (CDN, compression)

■ Backend Investigation

- • APM tool (Datadog/New Relic) — identify slowest API endpoints
- • Check DB query performance: slow query logs
- • Check for N+1 query problems in application code

■ Infrastructure Layer

- • CPU/Memory on app servers — is the host under pressure?
- • Check network latency between services (especially cross-AZ calls)
- • Load balancer: are requests distributed evenly?

■ Quick Wins

- • Enable CDN caching for static assets (CloudFront)
- • Add DB indexes on frequently queried columns
- • Enable GZIP/Brotli compression on web server

■ *Interview tip: Start with 'I'd check APM traces first to narrow it to frontend/backend/DB' — structured thinking scores more than listing every tool you know.*

Q08 Logs are not generating. How will you debug this issue?

■ Application Side

- • Is logging enabled in app config? (log level set to NONE or ERROR accidentally?)
- • Is the logging library initialized correctly? Check app startup logs
- • Check if the log file path has write permissions: ls -la /var/log/app/

■ Log Agent / Shipper

- • Is Filebeat/Fluentd/Fluent Bit running? (kubectl get pods -n logging)
- • Check agent logs for errors: kubectl logs
- • Verify the log path in agent config matches actual log output path

■ Kubernetes

- • kubectl logs — if blank, app is not writing to stdout/stderr
- • App must log to stdout/stderr in containers — not to files inside container
- • Check if logging sidecar is present if file-based logging is used

■ Log Storage (ELK/CloudWatch)

- • Verify index pattern in Kibana — correct date range?
- • Check CloudWatch Log Group exists and agent has IAM permissions

■ *Interview tip: In containers, logs MUST go to stdout/stderr. If a fresher doesn't say this, it's a red flag for interviewers.*

Q09 Disk space is full in a production server. What will you do?

■ Immediate Assessment

- df -h — see which partition is full
- du -sh /* | sort -rh | head -20 — find largest directories
- du -sh /var/log/* | sort -rh — logs are usually the culprit

■ Quick Relief

- Clear old logs: truncate -s 0 /var/log/app/old.log
- Remove unused Docker images: docker image prune -a
- Remove stopped containers: docker container prune
- Clean package cache: apt-get clean / yum clean all

■ Kubernetes

- kubectl describe node — check disk pressure condition
- Check if PVC is full: kubectl get pvc
- Evict low-priority pods if node has disk pressure taint

■ Permanent Fix

- Set up log rotation (logrotate) to auto-manage log file sizes
- Add CloudWatch/Datadog alert for disk > 80%
- Expand EBS volume or add new PV if storage genuinely needs to grow

■ *Interview tip: Say 'I would NOT just delete files blindly — I first identify the source, then clean safely.' Shows production discipline.*

Q10 Your application is not reachable from the internet. What could be wrong?

■ **DNS Layer**

- nslookup domain.com — does it resolve to the correct IP?
- Check Route53 records — is the A record or CNAME pointing correctly?

■ **Load Balancer / Ingress**

- Is the load balancer healthy? Check AWS ALB target group health
- kubectl get ingress — verify ingress has an ADDRESS assigned
- Ingress controller running? kubectl get pods -n ingress-nginx

■ **Security Groups / NACLs**

- Is port 80/443 open in the security group for the ALB?
- Is the ALB's security group allowed to reach EC2/NodePort?
- NACL rules are stateless — check both inbound AND outbound

■ **Service & Pod**

- kubectl get svc — is the service type correct (LoadBalancer/NodePort)?
- Are pods running and endpoints registered? kubectl get endpoints

■ **SSL/TLS**

- Certificate expired? curl -Iv https://domain.com | grep expire
- ACM certificate attached to the correct listener?

■ *Interview tip: Go layer by layer: DNS → Load Balancer → Security Group → Service → Pod. Never jump to 'maybe the pod is down' without checking network first.*

Q11 Your load balancer is not distributing traffic properly. How will you fix it?

■ **Check Target Health**

- AWS Console: EC2 > Load Balancers > Target Groups — are all targets healthy?
- Unhealthy target = health check failing (wrong path, port, or app returning non-200)

■ **Health Check Config**

- Verify health check path (e.g., /health or /actuator/health) returns 200
- Check timeout and interval — too tight causes false unhealthy
- Ensure security group allows ALB → target traffic on health check port

■ **Sticky Sessions Issue**

- If sticky sessions enabled, traffic goes to same instance — disable if not needed
- AWS: Target Group > Attributes > Stickiness

■ **Algorithm**

- Default is round-robin — if pods have different capacities, use least-outstanding-requests
- In Kubernetes: check if Service sessionAffinity is set to ClientIP accidentally

■ **Cross-AZ**

- Enable cross-zone load balancing to prevent AZ imbalance

■ *Interview tip: Sticky sessions causing uneven distribution is a very common real-world issue. Mentioning it sets you apart.*

Q12 Secrets got exposed accidentally. What is your immediate action?

■ **Immediate Response (First 15 mins)**

- Rotate/revoke the exposed secret IMMEDIATELY — before investigation
- AWS: IAM > Access Keys > Deactivate exposed key
- Revoke API tokens, DB passwords, OAuth secrets

■ **Contain the Blast Radius**

- Check CloudTrail for any API calls using the exposed key
- Check if any unauthorized resources were created
- Temporarily restrict access in security group if active exploitation suspected

■ **Git History Cleanup**

- git filter-branch or BFG Repo Cleaner to remove secret from git history
- Force push to remote AFTER rotation — never before
- Add the file to .gitignore immediately

■ **Root Cause & Prevention**

- Use Kubernetes Secrets + seal with Sealed Secrets or AWS Secrets Manager
- Enable GitGuardian or truffleHog in CI pipeline for secret scanning
- Use IRSA (IAM Roles for Service Accounts) to eliminate hardcoded AWS creds
- Never log environment variables

■ *Interview tip: ROTATE FIRST. Always say this first. An interviewer wants to hear that your instinct is to revoke, not investigate — investigation comes second.*

Q13 Auto Scaling is not working. How do you troubleshoot?

■ AWS Auto Scaling Group

- Check ASG Activity History in console — any scaling events fired?
- Is the CloudWatch alarm for CPU/custom metric in ALARM state?
- Check if the alarm threshold was actually crossed

■ Scaling Policy

- Is the policy attached to the ASG? aws autoscaling describe-policies
- Is cooldown period too long? (prevents rapid scale-up after scale-out)
- Is max capacity already reached? ASG won't scale beyond max

■ Kubernetes HPA

- kubectl get hpa — check TARGETS column (current vs desired)
- Is metrics-server running? kubectl get deployment metrics-server -n kube-system
- HPA requires metrics-server — if missing, HPA shows and won't scale

■ Instance Launch Issues

- AMI ID invalid or deleted
- Launch template has wrong instance type (not available in AZ)
- IAM role for EC2 instances missing

■ *Interview tip: HPA not working because metrics-server is missing is extremely common. Bring this up — most freshers don't know it.*

Q14 Monitoring alerts are not triggering. What will you check?

■ Alert Configuration

- Is the alert rule actually saved and enabled?
- Correct metric name? Even a typo in the metric name silences alerts
- Threshold correct — is the condition ever being met?

■ Data Source

- Is the metric actually being collected? Check Prometheus targets / Datadog agent
- Run the query manually in Grafana/Prometheus — does it return data?
- kubectl get pods -n monitoring — are Prometheus/Datadog pods running?

■ Notification Channel

- PagerDuty/Slack/Email integration working? Test with a manual alert
- Check if webhook URL is still valid — tokens expire
- Is the notification silenced/muted (maintenance window active)?

■ Datadog Specific

- Check agent status: datadog-agent status
- Is the DaemonSet running on all nodes? kubectl get ds -n datadog
- Check API key is valid and not rate-limited

■ *Interview tip: Always validate the metric collection first, then the alert rule, then the notification channel — in that order.*

Q15 Your Terraform deployment failed. How do you debug?

■ Read the Error

- Terraform prints the exact resource and error — read it fully before acting
- terraform plan -out=plan.out first to catch config errors before apply

■ Common Errors

- Provider auth failure: AWS credentials not configured or expired
- Resource already exists: imported externally — use terraform import
- Dependency cycle: resource A depends on B which depends on A
- Invalid AMI ID or instance type for region

■ State Issues

- terraform state list — see what's in state
- If partial apply: terraform show to inspect current state
- Use terraform taint to force re-creation if needed

■ Debugging

- TF_LOG=DEBUG terraform apply — verbose output showing API calls
- Check .terraform directory — providers downloaded correctly?

■ *Interview tip: TF_LOG=DEBUG is your best friend in Terraform debugging. Mentioning it shows hands-on experience.*

Q16 State file is corrupted. What will you do?

■ Don't Panic — Check Backup First

- S3 backend with versioning enabled — retrieve previous valid version
- aws s3api list-object-versions --bucket --key terraform.tfstate
- Download the last known good version and restore

■ If No Backup Exists

- terraform state pull — try to get the current state
- Manually reconstruct state using terraform import for each resource
- This is painful — reinforces why remote backend + versioning is mandatory

■ Immediate Prevention

- Always use remote backend (S3 + DynamoDB for locking)
- Enable S3 versioning on the state bucket
- Enable DynamoDB state locking to prevent concurrent modifications

■ State Locking Issue

- If apply failed and left a lock: terraform force-unlock
- Only do this if you're certain no other apply is running

■ *Interview tip: If you can't answer the fix, at least say 'This is why I always enable S3 versioning and DynamoDB locking.' It shows you've thought about prevention.*

Q17 Network connectivity issue between services. How will you analyze?

■ Basic Connectivity

- kubectl exec -it -- curl http://:/health
- nslookup from inside pod — does DNS resolve?
- ping or telnet to target IP/port

■ Kubernetes Networking

- kubectl get svc — correct ClusterIP and port?
- kubectl get endpoints — are pods registered as endpoints?
- If no endpoints, pod selector labels don't match service selector

■ Network Policies

- kubectl get networkpolicy — is there a policy blocking the traffic?
- Network policies are default-deny if applied — ensure egress/ingress rules allow the path

■ AWS Level

- Check Security Groups: does the source SG have outbound rule to destination SG?
- Is the VPC peering or Transit Gateway route table correctly configured?
- Check NACL: stateless — both directions must be explicitly allowed

■ *Interview tip: Start from inside the pod with curl/nslookup. Many candidates jump straight to security groups without first confirming DNS works.*

Q18 Your Jenkins job is stuck. How do you resolve it?

■ Check the Console Output

- Jenkins UI: Open the stuck build > Console Output
- Look for 'Waiting for executor', 'waiting for input', or a hanging command

■ Agent / Executor Issue

- Is the agent node online? Jenkins > Manage Nodes
- Agent disconnected mid-build — reconnect agent or restart Jenkins agent service
- All executors busy? Either free an executor or add more agents

■ Hanging Build

- Kill the stuck build from Jenkins UI (X button) — don't let it block the queue
- SSH into agent and kill the zombie process: ps aux | grep

■ Lock/Resource Contention

- Is the job waiting on a shared resource (Lockable Resources plugin)?
- Check if another build holds the lock — release manually if safe

■ Workspace Issue

- Full disk on Jenkins agent causes builds to hang silently
- Clean workspace: df -h on the agent node

■ *Interview tip: Mention that stuck jobs are often caused by an agent going offline mid-run, not a code issue. Shows real-world exposure.*

Q19 Multiple pods are restarting frequently. What could be the issue?

■ Get Restart Count

- kubectl get pods — RESTARTS column shows count
- kubectl describe pod — check Last State and Exit Code

■ Common Root Causes by Exit Code

- Exit 0 or 1: App crash — check kubectl logs --previous
- Exit 137 (OOMKilled): Memory limit too low — increase resources.limits.memory
- Exit 143: App received SIGTERM — preStop hook may be missing

■ Liveness Probe Misconfiguration

- Too aggressive liveness probe kills healthy pods — check probe timing
- kubectl describe pod | grep -A10 Liveness
- Increase initialDelaySeconds if app takes time to start

■ Resource Starvation

- kubectl top pods — are they hitting CPU/memory limits?
- Check if QoS class is Burstable or BestEffort — pods get evicted first
- Set both requests AND limits to get Guaranteed QoS

■ *Interview tip: OOMKilled (Exit 137) + missing resource limits is the #1 cause of CrashLoopBackOff in prod. If you say this confidently, you stand out immediately.*

Q20 High latency between services. How will you investigate?

■ Identify the Affected Path

- Use distributed tracing (Datadog APM / Jaeger) — find which service-to-service call is slow
- Check p99 latency, not average — averages hide tail latency problems

■ Network Layer

- Are services in the same AZ? Cross-AZ calls add 1-3ms but compound under load
- Check if DNS resolution is slow: time nslookup
- Network policies causing retries?

■ Database

- Slow DB queries are the #1 cause of high service latency
- Check slow query log: SET slow_query_log = ON; (MySQL)
- Missing index on JOIN or WHERE column

■ Application

- Synchronous calls that should be async (e.g., sending email inline)
- Connection pool exhausted — threads waiting for DB connection
- No caching on expensive repeated reads (add Redis/ElastiCache)

■ Kubernetes

- kube-proxy mode: iptables vs IPVS — IPVS is faster at scale
- DNS cache (NodeLocal DNSCache) reduces lookup overhead under high traffic

■ *Interview tip: 'I'd start with distributed traces to find the hot path before touching any config.' This is a senior-level instinct — use it.*
